

EXPERIMENT – 4

Title

To Make Use of Different SQL Clauses viz. WHERE, GROUP BY, HAVING, ORDER BY, UNION and INTERSECTION

Aim / Objective

To understand and implement various SQL clauses such as **WHERE, GROUP BY, HAVING, ORDER BY, UNION, and INTERSECTION** for retrieving, filtering, grouping, sorting, and combining data from database tables.

Theory

SQL provides various clauses that help users retrieve and manipulate data efficiently. These clauses are used to filter records, group data, sort results, and combine the output of multiple queries.

The commonly used SQL clauses in this experiment are:

- **WHERE** – Filters records based on a specified condition.
- **GROUP BY** – Groups rows having the same values.
- **HAVING** – Filters grouped records.
- **ORDER BY** – Sorts records in ascending or descending order.
- **UNION** – Combines the results of two or more SELECT statements and removes duplicate rows.
- **INTERSECT** – Returns only the common records from two SELECT statements.

Note: Oracle supports the **INTERSECT** operator directly. MySQL does not support **INTERSECT**. In MySQL, similar results can be achieved using **INNER JOIN** or **EXISTS**.

Learning Objectives

After completing this experiment, students will be able to:

- Retrieve records using conditions.
- Group records using GROUP BY.
- Filter grouped data using HAVING.
- Sort records using ORDER BY.
- Combine query results using UNION.

- Retrieve common records using INTERSECT.
-

Software Requirements

- Oracle Database
 - Oracle SQL Developer / SQL*Plus
-

Algorithm / Procedure

1. Open Oracle SQL Developer.
 2. Connect to the database.
 3. Create the Employee table.
 4. Insert sample records.
 5. Execute SQL queries using different clauses.
 6. Observe the results.
 7. Stop.
-

Program

Step 1 : Create Employee Table

```
CREATE TABLE Employee
(
EmpID NUMBER PRIMARY KEY,
EmpName VARCHAR2(30),
Department VARCHAR2(30),
Salary NUMBER(10,2)
);
```

Step 2 : Insert Sample Records

```
INSERT INTO Employee VALUES(101,'Rahul','IT',35000);
INSERT INTO Employee VALUES(102,'Priya','HR',42000);
INSERT INTO Employee VALUES(103,'Aman','Finance',39000);
INSERT INTO Employee VALUES(104,'Sneha','Marketing',45000);
INSERT INTO Employee VALUES(105,'Karan','IT',50000);
INSERT INTO Employee VALUES(106,'Riya','HR',38000);
```

```
COMMIT;
```

WHERE Clause

Program 1 : Display Employees Working in IT Department

```
SELECT *  
FROM Employee  
WHERE Department='IT';
```

Expected Output

EmpID	EmpName	Department	Salary
-------	---------	------------	--------

101	Rahul	IT	35000
-----	-------	----	-------

105	Karan	IT	50000
-----	-------	----	-------

ORDER BY Clause

Program 2 : Display Employees in Ascending Order of Salary

```
SELECT *  
FROM Employee  
ORDER BY Salary;
```

Program 3 : Display Employees in Descending Order of Salary

```
SELECT *  
FROM Employee  
ORDER BY Salary DESC;
```

Expected Output

EmpName	Salary
---------	--------

Karan	50000
-------	-------

Sneha	45000
-------	-------

Priya	42000
-------	-------

Aman	39000
------	-------

Riya	38000
------	-------

Rahul	35000
-------	-------

GROUP BY Clause

Program 4 : Display Department-wise Total Salary

```
SELECT Department,  
SUM(Salary) AS Total_Salary  
FROM Employee  
GROUP BY Department;
```

Expected Output

Department Total Salary

IT	85000
HR	80000
Finance	39000
Marketing	45000

HAVING Clause

Program 5 : Display Departments Having Total Salary Greater Than 70000

```
SELECT Department,  
SUM(Salary) AS Total_Salary  
FROM Employee  
GROUP BY Department  
HAVING SUM(Salary)>70000;
```

Expected Output

Department Total Salary

IT	85000
HR	80000

UNION Operator

Program 6 : Combine IT and HR Employees

```
SELECT EmpName  
FROM Employee  
WHERE Department='IT'
```

UNION

```
SELECT EmpName  
FROM Employee  
WHERE Department='HR';
```

Expected Output

Employee Name

Rahul

Karan

Priya

Riya

INTERSECT Operator (Oracle)

Program 7 : Display Common Employees

```
SELECT EmpName  
FROM Employee  
WHERE Salary>35000
```

INTERSECT

```
SELECT EmpName  
FROM Employee  
WHERE Department='HR';
```

Expected Output

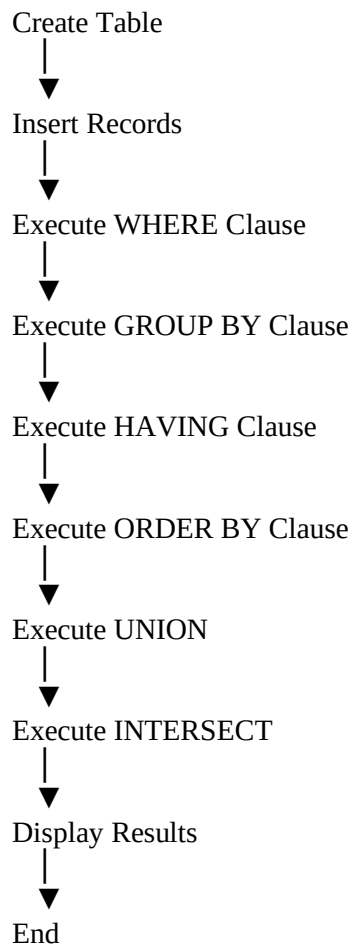
Employee Name

Priya

Riya

Flowchart

Start
↓
▼



Explanation

WHERE

Filters records according to specified conditions.

GROUP BY

Groups rows with similar values.

HAVING

Filters grouped records after aggregation.

ORDER BY

Sorts query results in ascending (default) or descending order.

UNION

Combines results from multiple SELECT statements and removes duplicate rows.

INTERSECT

Returns only the common rows between two SELECT statements.

Observation

- Records were filtered successfully using the WHERE clause.
 - Department-wise grouping was performed using GROUP BY.
 - HAVING filtered grouped data correctly.
 - Records were sorted using ORDER BY.
 - UNION combined records without duplicates.
 - INTERSECT returned common records successfully.
-

Result

Different SQL clauses including **WHERE**, **GROUP BY**, **HAVING**, **ORDER BY**, **UNION**, and **INTERSECT** were successfully implemented to retrieve, filter, group, sort, and combine data from the Employee table.

Applications

- Employee Salary Reports
 - Student Result Analysis
 - Sales Reports
 - Banking Applications
 - Inventory Management Systems
 - Hospital Management Systems
 - Payroll Systems
-

Advantages

- Efficient data retrieval.
 - Easy filtering of records.
 - Better report generation.
 - Supports data analysis.
 - Reduces query complexity.
-

Viva Questions

1. What is the purpose of the WHERE clause?
 2. What is the difference between WHERE and HAVING?
 3. Why is GROUP BY used?
 4. What is an aggregate function?
 5. What is the default sorting order of ORDER BY?
 6. What is the difference between UNION and UNION ALL?
 7. What is the INTERSECT operator?
 8. Does MySQL support INTERSECT?
 9. Can HAVING be used without GROUP BY?
 10. What is the execution order of SQL clauses?
-

Lab Assignment

1. Display employees whose salary is greater than ₹40,000.
2. Display employee names in alphabetical order.
3. Display department-wise average salary.
4. Display departments having more than one employee.
5. Combine records of IT and Finance departments using UNION.
6. Find employees who belong to HR and have a salary greater than ₹35,000.
7. Display department-wise maximum salary.
8. Compare WHERE and HAVING with suitable examples.

9. Explain the difference between UNION and UNION ALL.
 10. Write SQL queries using all the clauses covered in this experiment.
-

Precautions

- Use the **WHERE** clause before **GROUP BY**.
 - Use the **HAVING** clause only after **GROUP BY**.
 - Ensure that all **SELECT** statements in a **UNION** have the same number of columns and compatible data types.
 - Remember that **INTERSECT** is supported in Oracle but not directly in MySQL.
 - Verify the output after executing each query.
-

Learning Outcome

After completing this experiment, students will be able to:

- Retrieve records using conditional queries.
- Group and summarize data using aggregate functions.
- Filter grouped data using the **HAVING** clause.
- Sort records in ascending and descending order.